

DA3408 Assignment 2 Writeup

Question 1: Single-Stage vs. Multi-Stage Docker

I used `python:3.12-slim` for both Dockerfiles so that the only difference is single-stage vs. multi-stage. The naive image was 769 MB and the multi-stage image was 619 MB. This is a reduction of 19.5%. Both containers served `/healthz` and `/predict` correctly. These sizes are from the Git tag `q1`.

The installed libraries are the same size in both images (317 MB and 318 MB). The naive image is bigger because of what gets left behind. In the naive image, `pip install` leaves its 74 MB download cache in `/root/.cache`, and it ends up in the same layer as the libraries. It cannot be removed later because Docker layers cannot be changed after they are created. The naive image also uses `COPY . .`, which copies the whole folder, including `.git` and the screenshots. The build context was 880 kB for the naive image and 165 B for the multi-stage one. In the multi-stage build, `pip` installs into `/opt/venv` with `--no-cache-dir` in the builder stage. The final stage only copies `/opt/venv`, `predictor_app.py` and `model.joblib`.

Question 2: Docker Compose with Redis

Before predicting, the API now checks Redis using the SHA-256 hash of the input text as the key. On a hit, it returns the saved label without running the model. On a miss, it runs the model and saves the label with a 300 second TTL. The response is still `{"label": ...}`. I added an `X-Cache` header to show whether the request was a HIT or a MISS. The cache is only used if `REDIS_HOST` is set, so the same image also works in Question 4 without Redis.

The compose file has two services: `api`, built from the multi-stage Dockerfile, and `cache`, using `redis:7-alpine`. The API connects using `REDIS_HOST=cache`. Inside the API container, `cache` resolved to `172.20.0.2`. Sending the same message twice gave MISS and then HIT, with the correct label both times. My benchmark script sends 20 different messages and then sends the same 20 again. Misses averaged 2.12 ms and hits averaged 1.11 ms, so hits were about 1.9 times faster. The gap is small in absolute terms because this model is already very fast. I used an average instead of one request because the very first request is slow due to warm-up, which makes the speedup look bigger than it really is.

A Dockerfile only builds one image. It cannot start a second container, connect the two containers, or control the order they start in. Docker Compose does this for multiple containers, but only on one machine. Kubernetes does the same job across a cluster of machines, and it also handles scaling, self-healing and rolling updates.

Question 3: Kubernetes Indexed Job

I generated 8 shards of 100 signup records each, using a fixed seed. In each shard, 3 to 12 rows are broken, either with a bad email or with an empty field. There are 51 bad rows in total, and I know the count for each shard. Each pod reads `JOB_COMPLETION_INDEX`, checks that shard, and prints the result as JSON. The shards are copied into the image. The check itself takes a few milliseconds, so each pod waits 20 seconds (`HOLD_SECONDS`) after it finishes. Without this wait, the pods would finish before I could see them running at the same time.

Choosing parallelism. Each pod is a single-threaded Python script, so it cannot use more than one core, and checking 100 rows needs much less than that. I gave each pod 500m CPU and set the request equal to the limit. The cluster has 2 nodes with 2 CPUs each, which is 4000m in total. Kubernetes' own system pods already request 850m on the first node and 100m on the second node. I checked these values with `kubectl describe nodes`. This leaves 1150m and 1900m free. A pod has to fit on one node, so the first node can run 2 pods and the second can run 3. At most 5 pods can run at the same time. With 8 shards, a parallelism of 4, 5, 6 or 7 all take 2 rounds. A parallelism of 8 would take 1 round, but it would need 4000m and only 3050m is free. I chose 4 because it is the smallest value that still finishes in 2 rounds. It also places 2 pods on each node. One more detail: minikube reported my laptop's 16 cores as each node's CPU, so I reserved 14 of them with a kubelet setting. This makes each node have exactly 2 allocatable CPUs, as the question says.

Manifest and results. `job.yaml` uses `completions: 8`, `parallelism: 4`, `completionMode: Indexed` and `restartPolicy: Never`. The CPU request and limit are both 500m. It uses the Downward API to set `POD_NAME` and `NODE_NAME`. `kubectl get pods -o wide` showed 4 pods running at the same time, 2 on each node, in both rounds. `collect_results.py` reads each pod's logs through the Kubernetes API:

Shard	0	1	2	3	4	5	6	7
Invalid rows	3	4	5	8	5	10	12	4

These counts match what the generator created, and pods ran on both nodes. I did not use a shared volume because minikube's default storage provisioner creates `hostPath` volumes. A `hostPath` volume is a folder on one node's disk, so a pod on the other node cannot use it. Logs can be read through the API from any node, so storage is not needed.

With 3 nodes (6 CPUs), I would change the parallelism to 8. The first node still has 1150m free, and the two other nodes would have 1900m free each. That is room for $2 + 3 + 3 = 8$ pods, so all 8 shards could run in one round instead of two.

Question 4: Deployment, Self-Healing and Rolling Update

The Deployment runs 2 replicas of `spam-api:v1` with a CPU request of 250m and a limit of 500m. It has a readiness probe on `/healthz`. A NodePort Service exposes it on port 30080. One pod ran on each node, and requests through the Service returned correct responses.

Self-healing. I deleted the pod `spam-api-799798f56d-7w9fv`. A new pod, `spam-api-799798f56d-blvmq`, was running about 11 seconds later. It has the same `799798f56d` part in its name, which shows that it came from the same ReplicaSet. The *ReplicaSet controller* is responsible for this. The Deployment creates a ReplicaSet, and the controller keeps comparing the number of running pods with `replicas: 2`. When one pod was deleted, the count became 1, so the controller created a new pod.

Rolling update. For v2, I added `"version": "v2"` to the `/healthz` response and built it as `spam-api:v2`. I then ran `kubectl set image`. `rollout status` showed that the rollout completed, and `rollout history` showed revisions 1 and 2. During the update, I sent a request to `/healthz` every half second. The responses changed from v1 to v2, and every response returned 200. Kubernetes starts a new pod and waits until it is ready before removing an old one, so there are always 2 pods serving requests. My first attempt had one failed request because an old pod shut down before traffic had stopped going to it. I added a `preStop` hook that waits 5 seconds before the pod stops. The second attempt had no failures.

If the API had been deployed as a Job, nothing would keep 2 copies running. There would also be no rolling update and no rollout history. A Job cannot be updated to a new image, so I would have had to delete it and create a new one. The API would be down between those two steps.

Deployment vs. Job. A Deployment is for things that should run all the time, such as an API. It keeps the right number of pods running, replaces pods that fail, and updates without downtime. A Job is for work that has an end, such as the shard validation in Question 3. It runs until every task is done and then stops.